

Viro-AI Backend — Full Requirements & Implementation Prompt

PROJECT GOAL

Build a production-ready backend for the Viro-AI frontend that ingests multiple data sources, maintains data pipelines and storage, computes analytics & AI-driven insights, exposes secure REST/GraphQL APIs for the frontend, supports multi-tenant RBAC, and is scalable, observable, and auditable.

HIGH-LEVEL ARCHITECTURE & COMPONENTS

- API layer: REST + optional GraphQL, OpenAPI spec, request validation.
- Auth & IAM: OAuth2 / OpenID Connect, JWTs, RBAC, optional 2FA.
- Ingestion / Connectors: GA4, Facebook Ads, CSV, S3, Postgres, MySQL, custom API.
- ETL / Data Pipeline: ingestion → normalization → validation → enrichment → storage.
- Data Storage: PostgreSQL, ClickHouse/BigQuery for analytics, S3 for raw files.
- Insights/AI Engine: anomaly detection, trend analysis, recommendations.
- Job Scheduling: Airflow or cron for refreshes, model retraining.
- Observability: Prometheus, Grafana, ELK/Loki, OpenTelemetry tracing.
- Security: Encryption, audit logs, SSO, role-based access.
- CI/CD: Terraform infra, GitHub Actions, Kubernetes deployment.

NON-FUNCTIONAL REQUIREMENTS

- Multi-tenancy with isolation.
- API latency < 300 ms for simple queries.
- 99.9% uptime target.
- Scalability for ingestion workers.
- Fully auditable changes and actions.

DATA MODEL (PostgreSQL Core Entities)

(users, organisations, memberships, data_sources, datasets, raw_ingest_events, aggregates, insights, audit_logs)

API DESIGN (OPENAPI OUTLINE)

Includes:

- Auth endpoints
- Organisation + membership management
- Data source connection, sync triggers
- Metrics listing + querying
- Dashboard endpoints
- Insights operations
- Audit and admin endpoints

INGESTION & ETL FLOW

1. Connector setup
2. Initial sync job
3. Normalization worker
4. Enrichment
5. Aggregation
6. Indexing
7. Insight runner
8. Notification triggers

INSIGHTS & ML SUBSYSTEM

- Supports anomaly detection (z-score, decomposition)
- Trend & seasonal analysis
- Correlation-based insights
- Text generation for explanations
- Model serving and scheduled training

SECURITY REQUIREMENTS

- HTTPS, HSTS
- JWT best practices
- RBAC enforcement on each resource
- OAuth2/OIDC SSO support
- Audit logs for all actions
- Encrypted secrets (KMS)
- GDPR-compliant deletion/export

OBSERVABILITY & OPS

- Metrics: API latency, queue depth, sync freshness
- Logs: structured JSON
- Dashboards: Grafana metrics dashboards
- Alerts: ingestion failures, anomaly spikes, queue backlogs
- Health endpoints (live/ready)

CI/CD & INFRA

- Terraform modules for VPC, DB, clusters, S3, secrets
- GitHub Actions for pipeline
- Environments: dev/stage/prod
- Helm or Cloud Run deployment
- Automated DB migrations

TESTING STRATEGY

- Unit tests for connectors + transformations
- Integration tests for ingestion flow
- End-to-end tests (upload → insights)
- Load tests (k6)
- Security tests (SAST, dependency scanning)

DELIVERABLES & ACCEPTANCE CRITERIA

1. Clean OpenAPI spec
2. Working API with org/user creation
3. Functional connectors (GA4 + CSV)
4. ETL pipeline writing aggregates
5. Insights engine producing anomalies
6. RBAC working end-to-end
7. Observability dashboards
8. CI/CD working
9. Documentation (README, schemas, runbooks)
10. Complete test coverage for critical flows

TASK TEMPLATES (Developer-Ready)

- A — API Skeleton & Auth
- B — CSV Connector & ETL
- C — Insights Runner

CODE SNIPPETS (Pseudo)

(Examples of ETL workers, anomaly detectors)

EXTRAS

- Example metadata fields for frontend
- Timezone normalization
- Sample schema preview endpoints